

Spatial-aware data partition for distributed memory parallelization of ANN search in multimedia retrieval

Guilherme Andrade*, Renato Ferreira, George Teodoro

Federal University of Minas Gerais Computer Science Department, Belo Horizonte, Brazil

ARTICLE INFO

Keywords:

Distributed ANN Search
Multimedia similarity search
Descriptor indexing
ANN Search Data Partitions

ABSTRACT

Content-based multimedia retrieval (CBMR) applications are becoming very popular in several online services which handles large volumes of data and are submitted to high query rates. While these applications may be complex, finding the nearest neighboring objects (multimedia descriptors) is typically their most time consuming operation. In order to address this problem, several recent works have proposed distributed memory parallelization of approximate nearest neighbors (ANN) search. These solutions employ a variety of ANN algorithms and different parallelization strategies. In this paper, we have identified the currently used parallelization strategies (Data Equal Split (DES) and Bucket Equal Split (BES)) and systematically evaluated their performance. We have also developed a framework to simplify the deployment of ANN algorithms in distributed memory machines with customized parallelization or data partition strategies. We further proposed a novel class of data partition/parallelization strategies that takes into account the data spatial proximity. Our approaches (SABES and SABES++) improves data locality and the system efficiency as compared to DES and BES. For instance, SABES++ achieved speedups of 4.2× and 1.8× on top of DES and BES, respectively, in the baseline case (40 nodes). Further, SABES and SABES++ also attained higher multi-node scalability and the gains vs DES and BES increase a larger number of nodes. SABES++ is 14.5× faster than DES when 160 nodes are used.

1. Introduction

Content-based multimedia retrieval (CBMR) applications, including image search engines, real-time song identification, and tagging of photos in social networks, are becoming very popular [1,2]. These applications are required to handle already large and continuously growing volume of data. While CBMR applications may have several phases, the most time consuming and critical operation found on those applications consists in finding similar objects to the query in a large database. CBMR applications represent objects (images, videos, songs, etc.) via high-dimensional feature vectors (descriptors), such as Scale Invariant Feature Transform (SIFT), Vector of Locally Aggregated Descriptors (VLAD), Gist, Bag-of-Features (BOW), and deep features (100–1000+ dimensions) [3–6]. Thus, the similarity search in this case corresponds to finding the vectors in the dataset that are closest to a query vector, using a given distance metric, as performed by algorithms that solve the k-nearest neighbors (k-NN) problem.

Due to the large number of descriptors and their high dimensionality, using the exhaustive k-NN is not an attractive or viable solution. Several efficient indexing data structures have been proposed to address this problem. Most of them rely on organizing the dataset spatially

to allow for the search to take place and compute distances only to a fraction (pruning) of the indexed objects. Examples of such data structures include kd-trees, k-means trees, and cover trees [7–9]. The main problem with these strategies in CBMR is that they are impacted by the “curse of dimensionality” [10] and their performance degrades as the data dimensionality grows.

Approximate nearest neighbors (ANN) search approaches were then proposed to index and search in large high-dimensional datasets for those applications able to trade performance for accuracy. While there is a diversity of ANN solutions in the literature, most of them are developed using a similar concept or strategy. They use data buckets to store descriptors close in space, and restrict the search to a limited number of buckets. This is similar to the strategy used by the data structures described above (kd-trees, k-means trees, etc.) in the exact search, but in the ANN case the exact answer is not required and pruning is effective even with high dimensional descriptors. The locality sensitive hashing (LSH) [11] organizes descriptors into hash entries (buckets), and restricts the search to entries to which the query is also hashed. The Inverted File with Asymmetric Distance Computation (IVFADC) [12] algorithm is built using the concepts of quantization to

* Corresponding author.

E-mail addresses: gandrade@dcc.ufmg.br (G. Andrade), renato@dcc.ufmg.br (R. Ferreira), george@dcc.ufmg.br (G. Teodoro).

reduce data dimensionality and an inverted file to store indexed points. The data dimensionality reduction is important to reduce the amount of memory used to store the data and the inverted file is employed to prune the search to a subset of the points. The quantization is computed using centroids learned from the input data to represent the actual data points. In this process, points may be identified by their closest centroid along with a distance from it. The inverted file, on the other hand, consists of an array of lists of points. In IVFADC each inverted file entry has an associated centroid and points are stored in the entry or list corresponding to their closest centroid. This allows for the search to be limited to a subset of the inverted file entries (buckets or lists of points) or those associated to the centroids closest to the input query, reducing the computational demands. Also, a number of other algorithms are available in the fast library for approximate nearest neighbors (FLANN) [9]. Here, we focus our attention to the IVFADC, because it has been shown to attain higher throughput than competitors while requiring a fraction of the memory to store the index in several settings.

The ANN strategies have been able to significantly improve performance of CBMR applications, but ANN design originally focused on sequential execution on a single machine. However, current applications deal with large datasets and high query rates and, consequently, the processing and storage demands tend to be greater than those available in a single machine. This observation has motivated the development of distributed memory parallelizations of ANN by several groups [9,13–18]. The parallelization strategy employed by these works is based on a Master–Worker model, but they differ in the way data is partitioned among Workers. A common approach is to divide descriptors evenly in a round-robin fashion with Workers and this strategy is called here Data Equal Split (DES). Another approach found in the literature partitions buckets among Workers and is called here Bucket Equal Split (BES).

Although widely used, DES and BES have not been systematically compared in the literature, which is the first contribution of this work. This is performed here by developing a framework for deploying ANN algorithms in distributed memory systems. Using this framework, we have implemented DES and BES with IVFADC. We have further proposed a novel data partition and parallelization approach in which buckets of points are split among Workers taking into account their spatial location. This strategy is called here Spatial-Aware Bucket Equal Split (SABES). It assigns buckets near in the space to the same Worker, as these buckets are more likely to be accessed together when answering a query. BES and SABES assign the same number of buckets to each Worker. However, buckets may have different number of points, creating data imbalance among Workers. This motivated the development of SABES++ that tries to distribute buckets near in space to the same Worker while, but also balances the number of data points indexed by Workers. We additionally investigated the use of data (bucket) replication to reduce the number of Workers involved in a search.

The experimental results have shown that BES, SABES, and SABES++ have attained superlinear scalability in weak scaling settings, whereas DES scales near to linear. The spatial-aware strategies (SABES and SABES++) led to higher throughput and better scalability, while the ability of SABES++ to balance the data indexed by Workers have resulted in a performance improvement of about 1.12× on average as compared to SABES. This paper extends our work [19] with a detailed description of the solution and related work, the proposal and evaluation of SABES++, and additional experiments to evaluate the impact of number of data partitions (centroids) to the parallelization strategy. The contributions of this work may be summarized as follows.

- We have proposed and implemented a framework targeting the development of ANN algorithms in distributed memory machines. It simplifies development of novel data partition/parallelization strategies with the implementation of only two routing functions;

- We have developed new data partition strategies and compared them to existing approaches. Our strategies divide data buckets (as defined by the ANN algorithm) in the distributed system taking into account data locality and balancing (SABES and SABES++). For instance, in an experiment with 40 compute nodes, SABES++ achieved a speedup 1.8× vs. BES that was the most competitive approach available in the literature.
- We have also carried out scalability evaluations in which algorithm search parameters are varied along with different data partition/parallelization strategies. These experiments have employed up to 160 computing nodes with a dataset of 16 billion SIFT vectors. The results have shown superlinear weak scaling efficiency for our proposals. As a result, the larger the number of nodes used, the higher tend to be the performance improvements attained by our data partition strategies.

The rest of the paper is organized as follows. Related work is discussed in Section 2, whereas the IVFADC algorithm used as our baseline ANN search indexing is presented in Section 3. Our proposed framework is detailed in Section 4, while the description of the data partition strategies is shown in Section 5. Experimental results are presented and discussed in Section 6 and we concluded and highlighted directions for future work in Section 7.

2. Related work

This section presents our closest related work on ANN search. We present an overview of popular and effective ANN indexing algorithms and we discuss main parallelization works in the domain. The distributed memory solutions are categorized according to: (i) algorithm implemented, (ii) the framework used in the implementation, (iii) the ability to maintain the index in-memory that is a requirement to answer queries in online applications, (iv) the data partition used, (v) the use or not of locality-aware data (index) distribution, and (vi) the employment of load balancing.

Approximate nearest neighbors (ANN) search was proposed to improve the scalability and speed of NN search in high-dimensional spaces for applications in which exact answers are not essential. Several competitive techniques and algorithms have been proposed [9,11,12,29,30]. The locality sensitive hashing (LSH) [11] is one of these approaches. It employs locality sensitive hashing functions to index and search in the high-dimensional space. In this case, it is expected that multimedia objects hashed to the same hash entry to be also located near in the space. The Multicurves [31] is an indexing that performs multiple projections of subsets of the descriptors' dimensions to an 1-dimensional space using space-filling curves. During the search phase, Multicurves executes the same projections using the query object descriptor, and uses them to retrieve the nearest points in the 1-dimensional sorted lists. The IVFADC algorithm [12] decomposes descriptors into a Cartesian product of lower dimensionality subspaces to further quantize them. It also uses an inverted file to organize data points and limits the search to a subset of the data. The fast library for approximate nearest neighbors (FLANN) [9] builds a framework that automatically selects the best strategy, among algorithms such as randomized kd-trees [32], hierarchical k-means [33] and LSH [11], for a given dataset. FLANN has been extensively used as a baseline for performance comparisons in the domain. IVFADC has been shown to deliver smaller query response time than most of the related work [12] while, at the same time, requires less memory. This is the reason for using IVFADC in this work. Therefore, it is presented in detail in Section 3.

Parallel and Distributed ANN Search gained attention in the recent years as a complementary approach to the algorithmic ANN improvements [9,13–18]. The work Pioneer [13] is a pioneer solution that uses MapReduce. In [13], the index is stored in the distributed file system, which is visible to all compute nodes. The search is then carried

Table 1
Comparison between parallel and distributed ANN search strategies.

Indexing algorithm	Framework	In-memory	Data partition	Locality-aware	Load balancing
LSH [13]	MapReduce [20]	×	BES	×	×
LSH [14]	DHT [21]	✓	BES	×	×
Index Tree [15]	MapReduce [20]	×	BES	×	×
eCP [22]	Spark [23]	✓	BES	×	×
LSH [2]	MPI [24]	✓	DES	×	×
FLANN [9]	MPI [24]	✓	DES	×	×
Multicurves [17]	Anthill [25]	✓	DES	×	×
IVFADC [18,26]	MPI [24]	✓	DES	×	×
SLASH [27]	MPI [24]	✓	DES	×	×
LC [28]	MPI [24]	✓	BES	×	✓
This work (IVFADC)	MPI [24]	✓	DES, BES SABES, SABES++	✓	✓

out independently in each node by reading all data required to answer an incoming query. This strategy is intensive in I/O and may present limited performance due to the lack of data locality. Some of these I/O bottlenecks due to the file system based storage were addressed in [14], which used an Active Distributed Hash Table (DHT) to store the index in a distributed memory. A recent work [27] has proposed the SLASH (Sketched Locality Sensitive Hashing) algorithm that extends LSH with the use of sketches to avoid distance computations in the retrieval process. The authors have shown that SLASH performs well with extremely high-dimensional and sparse datasets. The distributed memory implementation was built on top of MPI using DES.

The Faiss library [34] is another important tool in the domain. It implements a series of ANN algorithms targeting CPU and GPU based systems. In this work, the IVFADC indexing implementation used in each node of the distributed search strategies leverages Faiss. An implementation of the Index Tree similarity search for distributed memory on top of MapReduce is presented in [15]. It stores the index into a distributed file system, having the same limitation presented above. However, it implements an interesting query reordering optimization to reduce file system data accesses. Another interesting work [22] leveraged Spark [23] for the implementation of distributed memory ANN search. It stores the index in data structures called Resilient Distributed Datasets (RDD) to maintain it in-memory after first access. This approach is designed to deal with large query batches to amortize system overheads. These limitations motivated parallelizations that partition the index data among computing nodes of distributed memory systems and allow for fast querying required by online applications. These approaches were deployed with LSH [2], FLANN [9], Multicurves [17], and IVFADC [18,26]. All of them employed the DES partition strategy.

Recent works on ANN and related operations have carried out research load balancing [28,35,36]. In [28], it is implemented an algorithm called List of Clusters (LC) [37], and its parallelization takes place by distributing cluster lists among nodes. The strategy proposed replicates cluster lists in the distributed system to balance load. It does not consider spatial location on the cluster assignment and may also incur in significant overhead due to the data replication. Also, the LC algorithm has a higher memory demands as compared to IVFADC, because the latter stores only a compact representation of the data descriptors. Other works [35,36] have developed load balancing techniques for the All Pairs Similarity Search (APSS) operation, which identifies similar objects in a dataset. In these works, a weighted graph is used to represent relationship between data elements, and this relationship is used to reduce imbalance and minimize computation. Also, the authors of [38] have developed strategies to minimize load imbalance in the another related operation called k-NN Join. Their approach divides data buckets with a larger number of points to avoid overloading specific processes. While these optimizations were not

evaluated in ANN, we believe they are worth mentioning because they may inspire new solutions in the domain.

Table 1 summarizes the related work on distributed memory implementations of ANN search. As may be noticed, several algorithms have been parallelized with the use of different framework. Some solutions focus on batch query processing (offline), which is a characteristic of those that do not maintain the data in-memory. It is shown that BES and DES are popular choices for data partitioning, but, unfortunately, previous works have not compared them. Our solution is the first to implement and compare multiple data partition strategies, use locality-aware data partitioning, and perform load balancing (without replication) in ANN search.

3. Background: ANN search with quantization

This section presents basic concepts of product quantization and its use in approximate nearest neighbor search. A detailed description may be found in [12]. The main idea of the vector quantization is to map a high-dimensional vector x to a lower-dimensional space. Working with lower dimensionality data has the benefits of (i) reducing distance computation costs and (ii) smaller indexing storage requirements. Given an input x vector in a D -dimensional space, the quantization creates a lower-dimensional representation $q(x)$, where $q(x)$ is the nearest vector in $C = \{c_i; i \in \tau\}$ with τ being an index set with k reproduction values. The reproduction values in C are called centroids and C obtained in a training phase, for instance, executing a k-means in a data sampling. The C centroid set is called codebook. In essence, the quantization process organizes the points into k cells similarly to a Voronoi diagram.

The quantization presents different trade-offs. A large number of centroids enables to represent a vector with more precision, but it incurs in a higher computational cost to find the closest centroid to a input vector in the quantization. The product quantization is a strategy to handle this problem. It attains the behavior of a large number of centroids with multiple small centroid sets. This is achieved by splitting the input vector x into a set of subvectors to be quantized separately, and by combining quantizations to create a large number of quantization possibilities. Each subvector u is lower-dimensional, assuming m subspaces to be used, the subvectors $u_j, 1 \leq j \leq m$ is composed of $D^* = D/m$ dimensions. The product quantization is defined as follows:

$$\underbrace{x_1, \dots, x_{D^*}}_{u_1(x)}, \dots, \underbrace{x_{D^*+1}, \dots, x_D}_{u_m(x)} \rightarrow q_1(u_1(x)), \dots, q_m(u_m(x)) \quad (1)$$

The quantization of x is a Cartesian product of its quantized subvectors: $q(x) = q_1(u_1) \times q_2(u_2) \times \dots \times q_m(u_m)$. This combination of multiple small centroid creates the codebook $C = C_1 \times \dots \times C_m$. Where, each C_j contains k centroids, and k^m combinations exist, resulting in a

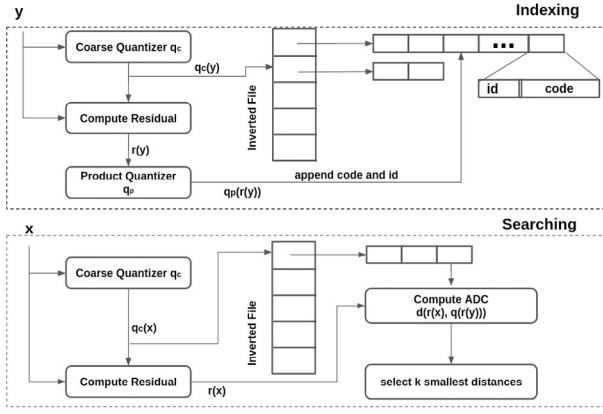


Fig. 1. IVFADC indexing and searching phases.

large codebook. After quantization, distances between a query vector x and the dataset computed in the ANN search will be carried out in the new quantized space. It was proposed two strategies to compute those distance in [12]: (i) Symmetric Distance Computation (SDC) that compute distances using a quantized version of the query ($q(x)$) and the quantized indexed dataset ($q(y)$); and, (ii) the Asymmetric Distance Computation (ADC) that computes the distances using the quantizers of the indexed data ($q(y)$) and the actual value of the query x . ADC attained better overall performance and is defined as.

$$\tilde{d}(x, y) = d(x, q(y)) = \sqrt{\sum_{j=1}^m d(u_j(x), q_j(u_j(y)))^2} \quad (2)$$

Even in a lower dimensional space computing an exhaustive search is impracticable. Thus, it was proposed the use of an inverted file system with ADC (IVFADC) to prune the search. This approach groups data elements quantized to the same centroid into inverted file entries, and restricts the search to a few inverted file entries associated to centroids nearest to the query.

Fig. 1 shows the indexing structure with the inverted file system, where each entry is represented by a “coarse quantizer” (centroid). Each inverted file entry contains a list of quantized points (id and code per point). The *id* can be a descriptor identification, for instance, the image from which it was computed. The *code* is used to improve the distance approximation in ADC, being the distance between the vector and the corresponding/nearest coarse centroid. Besides that, *code* is also quantized with product quantization to reduce storage demands.

Algorithm 1. Indexing and Searching with IVFADC

```

1: function Indexing( $y$ )
2:    $k' \leftarrow q_c(y)$ 
3:    $r_y \leftarrow y - C_c[k']$ 
4:    $code \leftarrow q_p(r_y)$ 
5:    $ivf[k'].append(y.id, code)$ 
6: function Searching( $x, w, k$ )
7:    $nnc \leftarrow kNN(C_c, x, w)$ 
8:   for  $i \in 1 \dots w$  do
9:      $r_x \leftarrow x - C_c[nnc[i]]$ 
10:     $distList \leftarrow dist(index[nnc[i]], r_x)$ 
11:     $ann \leftarrow k-select(ann, distList, k)$ 
12: return  $ann$ 

```

The IVFADC indexing and searching phases are presented in Algorithm 1. The indexing has the following steps:

1. Compute nearest coarse centroid to input vector y (line 2).
2. Calculate the residual value r_y of y to the k' -th (closest) coarse centroid ($C_c[k']$) (line 3)

3. Compute *code* by product quantizing the residual (line 4)
4. Store *id* and *code* in the inverted file entry associated to the nearest coarse centroid (line 5)

The searching phase receives as input the query vector (x), number of inverted file entries to visit (w), and number of nearest neighbors to retrieve (k), and executes the following steps:

1. Compute w nearest coarse centroids and store them in nnc
2. Search in the inverted file entries associated to each of these centroids (lines 8 to 11). For each entry:
 - (a) Compute the residual of x to the respective coarse centroid (line 9)
 - (b) Compute ADC distances to all points stored in the entry (line 10)
 - (c) Select the k nearest points and merge with those found in previously inverted file entries. (line 11)
3. After the w entries are visited, return results (ann).

4. Distributed memory IVFADC parallelization

The design and implementation details of our solution for ANN distributed memory parallelization is presented in this section. As previously discussed, our target is online applications in which minimizing the execution time of each query is important. Therefore, we have designed a distributed memory parallelization that maintains the index in memory for fast retrieval and communicates using Message Passing Interface (MPI) [24]. The proposed system has been designed to allow for quickly modifying the data partition strategy and to simplify implementation of other ANN algorithms in the future.

We can decompose the solution into the index building and searching phases, which are implemented using four types of processes: (i) Readers: responsible for reading and distributing the dataset; (ii) Coordinators (Co): that (a) receive user input queries and forward them to a specific set of Query Processor (QPs) and (b) compute the global k -nearest neighbor response; (iii) Query Streamers (QS): responsible for generating a query flow simulating the end-users; (iv) Query Processors (QPs): responsible for maintaining a local (IVFADC) index and to search on its index partition. Fig. 2 provides an overview of each phase and the components involved.

The *index building phase* has two main steps. The first one is responsible for creating a set of Coarse Centroids (C_c) used to define the inverted file lists in IVFADC. During this phase, we execute the k -means algorithm from a dataset sample (training dataset), creating as result a set of representative centroids or the codebook. This algorithm is executed for a subset of the dataset and our current implementation is a parallel shared-memory version, meaning that it is executed on a single node. We have not yet deployed it for distributed memory execution because k -means is executed before the execution (off-line) and, once created, the codebook is reused for future executions. Thus, the amortized cost of creating the codebook is small. Further, the second step distributes the input dataset among Query Processors (QP), which will locally index their data partition. Readers quantize input data descriptors before sending them over the network to the QPs, since this approach reduces the amount of data sent. The *sendTo* function implemented in the Reader (yellow block in Fig. 2) is responsible for choosing the QP instance(s) to which the descriptors are sent. This is one of the framework pieces that may be customized to implement different data partitions. This section assumes that round-robin is used in the data distribution. Consequently, input descriptors are equally divided among QPs (DES strategy).

The *searching phase* executes the actual nearest neighbors search using the following processes: (i) Query Stream (QS), (ii) Coordinators (Co), and (iii) Query Processors (QP). The QPs submits queries to the system, which are received by the Coordinators. The Co sends the

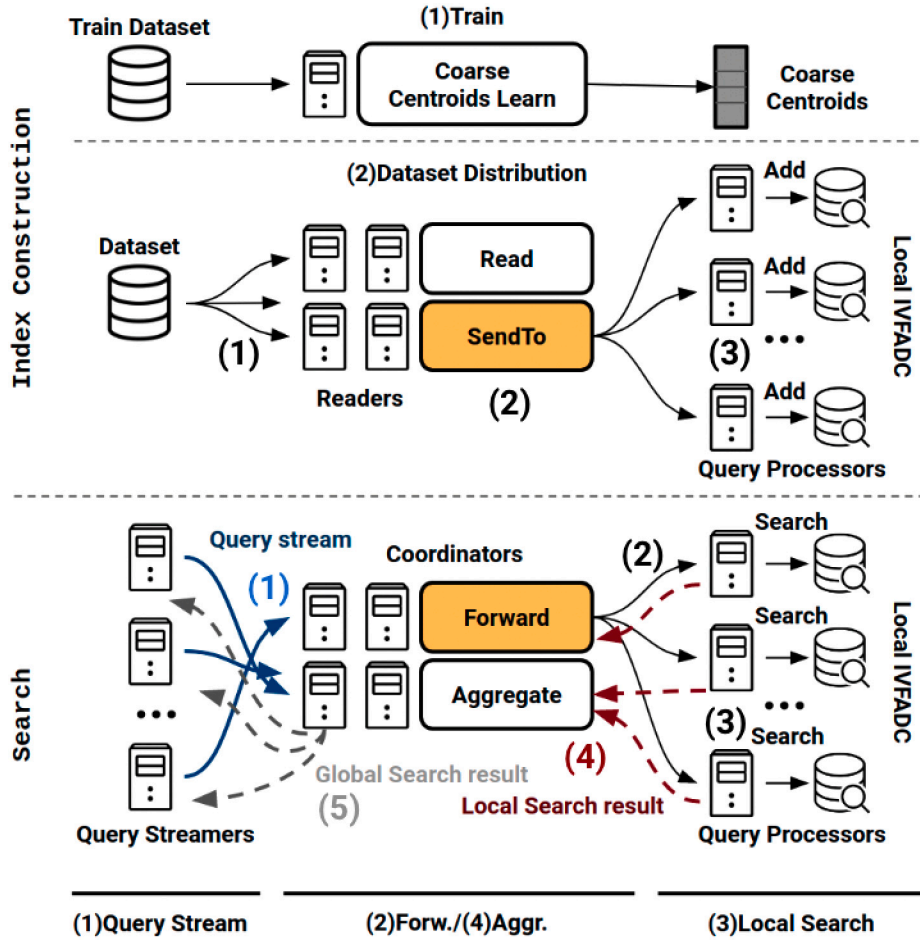


Fig. 2. The distributed memory system for indexing and searching using the IVFADC algorithm has two phases: index building and searching. The index building phase has two main steps: (i) Training to create or load centroids/codebook and (ii) Dataset distribution in which Reader processes distribute descriptors following the partitioning implemented by *SendTo* method (step 1), which are indexed by Query processors (QPs) (step 2). The searching phase starts with the Query Stream (QS) sending the query to Coordinators (step 1). Coordinators nodes, following the implemented 'Forward' method sends the query to the set of QPs involved in answering that request (step 2). QPs compute a local ANN in their local index (step 3) and send result back to the Coordinator for aggregating local nearest neighbors into a global nearest neighbors response (step 4). Finally, global results are send to the QS that represent the end-user here. The way descriptors are partitioned among QPs will impact their organization during query processing. This can be customized ("send to" and "forward") to implement and evaluate multiple data partition strategies as presented in Section 5.

query to the QPs responsible to process it, and will wait until the QPs local search results are returned to be aggregate them into a global response by the Co responsible for it. After aggregation, the Co returns the global response of each query to a single QS that submitted it to the system. This communication is carried out asynchronously, and the Co will continue executing other computations while the response is not received. The QPs instances that receive a query are computed by the query *forward* operation (yellow in the Coordinator), which is the second function that can be customized to implement a novel data partition strategy. Since descriptors were distributed in round-robin in the *sendTo*, each query must be sent to all QPs since descriptors in each inverted list entry are split among QPs. Our system supports the concurrent execution of indexing and searching phases as required in dynamic datasets.

The Coordinators and Query Processors are implemented as multithreaded processes using Pthreads (as illustrated in Fig. 3). The Coordinator uses three threads: (i) query receiver thread reads input queries and stores them in a queue of tasks (query queue); (ii) query forward retrieves a task from the query queue and executes the *sendTo* operation to decide the QPs that should cooperate to answer that request, and (iii) response aggregator that receives QPs local k-nearest neighbors and merges them into a global query response. The QPs are implemented with the following threads: (i) query receiver that is responsible for receiving query from Coordinators; (ii) local search that

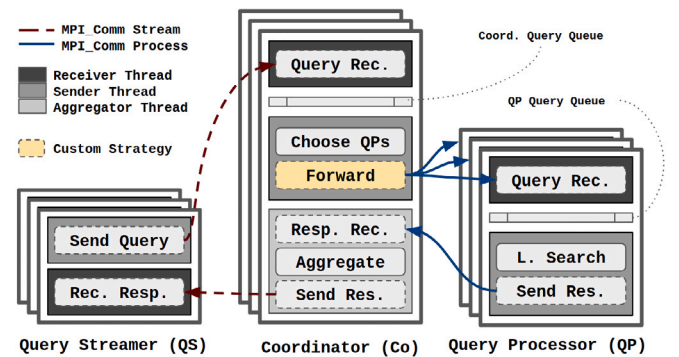


Fig. 3. Coordinators, Query Processors, and Query Streamers are implemented as multithreaded processes using Pthreads. The thread scheme used is executed in a Consumer/Producer way. This approach is important to not create internal bottlenecks given the constant flow of queries received by Coordinators and forwarded to Query Processors.

is responsible for performing the IVFADC search in the local index. This IVFADC search is executed by multiple (iii) working threads dispatched by the local search. Thus, a single QP is created per node to minimize the number of index partitions and, consequently, communication.

After local k-NN is computed, results are sent to the Co responsible for that particular query.

While the distributed system discussed in this section is presented in the context of the IVFADC algorithm, it is sufficiently generic to implement multiple ANN indexing strategies available in the literature. In this case, the indexing algorithm to be deployed should be instantiated locally with the QPs, and customizable message routing functions could be modified to compute the appropriate data partition and querying in each case. Thus, approaches implemented and evaluated in this work can be leveraged by other indexing algorithms.

5. Data partition and spatial organization strategies

The proposed distributed memory of IVFADC was built to make possible the deployment of different data partition strategies. This is achieved through customization of communication routing implemented by the *sendTo* and *forward* operations. As shown in Fig. 2, the *sendTo* implements the distribution of descriptors, and the *forward* operation is used in the Searching Phase. This section shows the customization of these operations to implement the following data partition strategies: Data Equal Split (DES), Bucket Equal Split (BES), and the new Spatial-Aware Bucket Equal Split (SABES), Spatial-Aware Bucket Equal Split with Data Balancing (SABES++) and Spatial-Aware Bucket Equal Split with data replication (SABES+r). An overview of the distribution performed in each case is presented in Fig. 4.

5.1. Data Equal Split (DES)

DES divides the input data feature vectors equally among the QP processes in a round-robin fashion during the index building phase (*sendTo*). Consequently, each QP has a complete IVF structure with all centroids (entries), while data (descriptors) are split among QPs. To ensure that all w nearest coarse centroids lists will be entire visited during search phase, queries must be forwarded from the Coordinators to all QPs, meaning that the *forward* is a broadcast. This strategy has been found in several previous works [2,9,18].

5.2. Bucket Equal Split (BES)

Most of the ANN indexing algorithms follow a common data organization pattern that consist of grouping feature vectors into data buckets, where objects stored in the same bucket are expected to be near in the space. Thus, another approach for partitioning the data is to divide data buckets among QPs. This strategy is called Bucket Equal Split (BES). In the search, it is expected that only a subset of the QPs will have to be consulted to answer a query, reducing communication traffic and computation startup overheads as compared to DES.

By using the IVFADC, inverted file entries are distributed equally among the QPs. This distribution ensures that all points within a given IVF entry will be stored in a single QP. Since each inverted file entry has an associated coarse centroid, this strategy corresponds to partitioning centroids and their closest points among QPs. Given that $|C_c|$ centroids are instantiated, the i -th QP ($QP_i, i \in 1 \dots n$) stores $|C_c|/|QP|$ centroids. Because centroids are distributed in a round-robin among QPs, the i -th QP will store centroids C_j for which $j \bmod |QP| == i$. This distribution is presented in Fig. 5 for a dataset of points represented in two dimensions. In this example, the 6 centroids in the codebook are distributed among the 2 QPs in round-robin along with their closest points in the dataset. In this case, a search query will have to be routed to each machine containing at least one of its w closest centroids.

The customized implementations of *sendTo* and *forward* are presented in Algorithm 2. During the Index Distribution phase, the *sendTo* receives a data point y to be indexed and calculates its nearest centroid k' or the inverted list entry in which y must be stored (line 2). The mapping from the centroid to the QP instance is then computed (line 3) and returned (line 4).

Algorithm 2. BES SendTo and Forward methods

```

1: function sendTo(y)
2:    $k' \leftarrow q_c(y)$ 
3:    $target \leftarrow k' \% n$ 
4:   return target
5: function forward(x, w)
6:    $nnc \leftarrow kNN(C_c, x, w)$ 
7:   for  $i \in 1 \dots w$  do
8:      $target \leftarrow target \cup nnc[i] \% n$ 
9:   return target

```

In the searching phase rather than sending the query vector x to all QPs as in the DES, we must find the set of QPs that store the w nearest coarse centroids to the query x . Thus, as shown in Algorithm 2, the *forward* computes the w nearest coarse centroids (line 6) and maps them to the respective QP instances (line 8). In fact, BES tends to use a smaller number of QPs than DES. It will send the query to no more than $\min(w, n)$, where n is the number of QP instances, and w is the number of buckets visited.

5.3. Spatial-Aware Bucket Equal Split (SABES)

SABES extends BES with a bucket distribution in which coarse centroids near in the space are more likely to be assigned to the same QP. The assumption for building SABES is that those data buckets close in the space have a high probability of being accessed by the same query. Consequently, it would reduce the number of QPs used to answer a query. SABES strategy main modification as compared to BES is that it partitions centroids into regions. This process is computed using a k-means on the centroids to separate them in clusters/regions. Further, instead of distributing individual buckets (centroids) among nodes as in BES, SABES distributes regions or clusters of centroids among nodes or QPs. The experimental results presented in Section 6.1 shown in practice that SABES indeed reduces the number of QPs used to answer a query.

Algorithm 3. SABES Region Construction, SendTo and Forward methods

```

1: function regionConstruction( $C_c, n$ )
2:    $R \leftarrow k\text{-means}(C_c, n)$ 
3:   return  $R$ 
4: function sendTo(y)
5:    $k' \leftarrow q_c(y)$ 
6:    $target \leftarrow getRegion(R, k')$ 
7:   return target
8: function forward(x, w)
9:    $nnc \leftarrow kNN(C_c, x, w)$ 
10:  for  $i \in 1 \dots w$  do
11:     $target \leftarrow target \cup getRegion(R, nnc[i])$ 
12:  return target

```

SABES has an additional off-line or pre-processing step in which centroids C_c are grouped into n regions R , and each region R_i is attributed to a QP_i . These regions are computed using the k -means algorithm having the coarse centroids as input to the method *regionConstruction*(C_c, n) in Algorithm 3. The return R is a mapping (array) with information of the region in which each coarse centroid is located, which is further used in the *sendTo* and *forward* methods. The *sendTo* and *forward* operations are also presented in Algorithm 3 and are similar to those in BES. The main change is the function *getRegion* that maps a coarse centroid id to the region R_i . This function simply accesses the map R indexed by the centroid id and returns its respective region. The SABES data partition is also exemplified in Fig. 5. Differently from BES, SABES distributes regions among QPs. Because

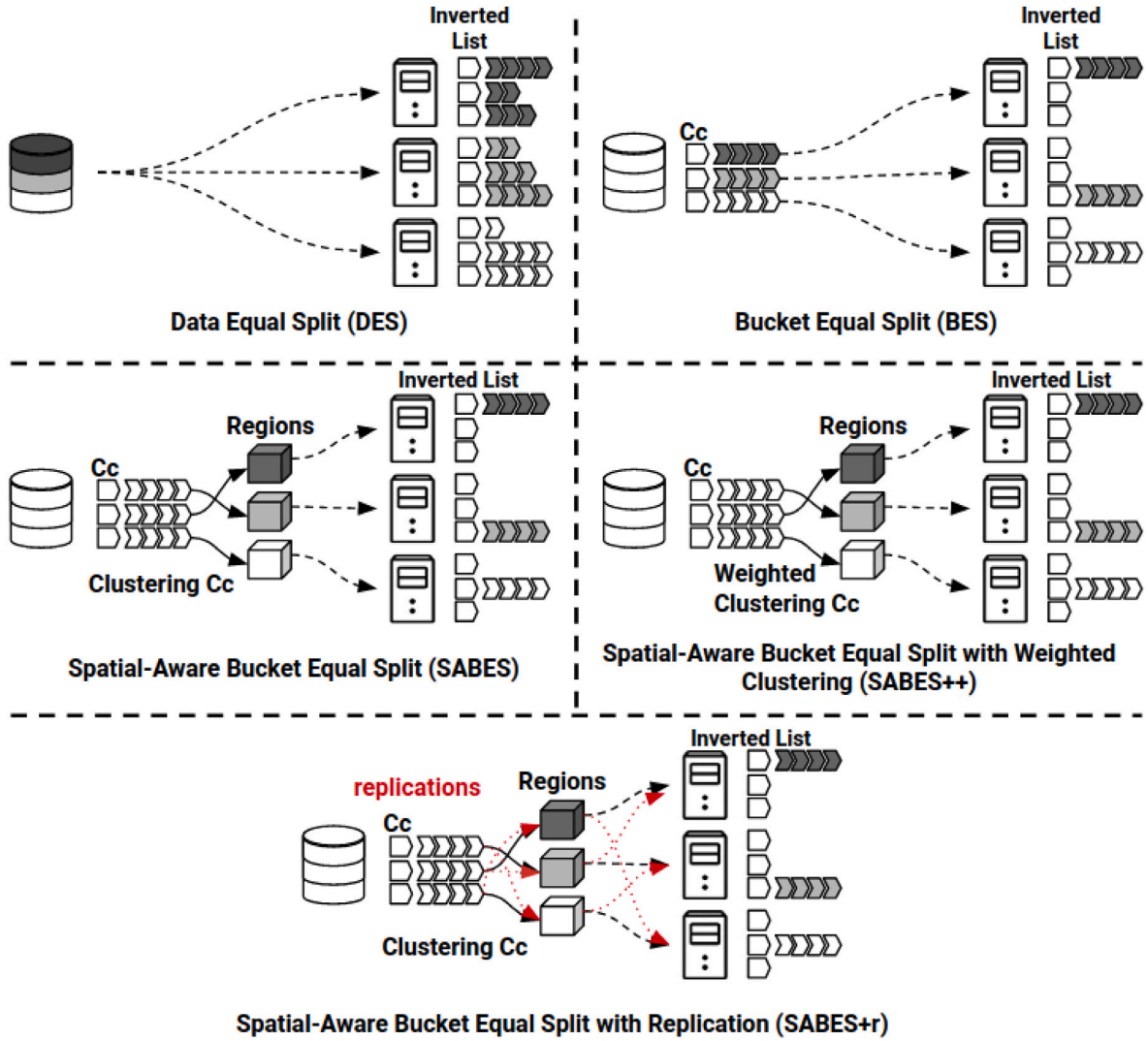


Fig. 4. Data Partition Strategies: (i) DES: dataset are equally divided among QPs; (ii) BES: distributes the inverted file entries among the QP; (iii) SABES: distributes inverted file entries among QPs taking into account their spatial location; (iv) SABES++: distributes inverted file entries among QPs taking into account their spatial location and data balancing. (v) SABES+r: includes inverted file entries replication into r nearest regions.

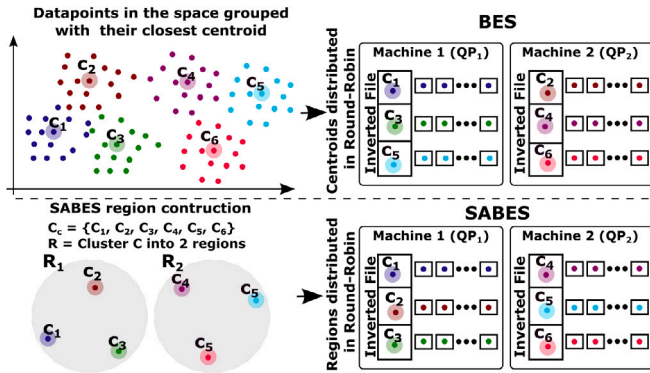


Fig. 5. Data Partition Strategies: BES vs SABES data partitions. BES distributes centroids and associated data points in round-robin among QPs, while SABES distributes regions (set of centroids) among QPs.

the regions are created by grouping centroids close in the space and queries need to access their closest centroids, we expect that lower averaged number of QP nodes/processes will have to be consulted

during the search. As previously mentioned, this occurs experimentally (Section 6.1).

5.4. Spatial-Aware Bucket Equal Split with data balancing

The Spatial-Aware Bucket Equal Split with Data Balancing (SABES++) has been proposed to extend SABES with an additional partitioning step that reduces data imbalance among QPs. The problem with SABES is that it assigns the same number of buckets (inverted file list entries) per QP instance, but the amount of data (descriptors) stored with different buckets (centroids) may diverge significantly. Consequently, the set of centroids assigned to a given QP instance may lead to widely differing storage demands when compared to other instances, which results in data and load imbalance.

In order to mitigate the imbalance with SABES, SABES++ has proposed a novel strategy to organize coarse centroids into regions. In this approach, coarse centroids are grouped considering their spatial location and the number of points associated to them. The goal is to group centroids near in space, but to avoid imbalance among clusters of centroids (regions). This has been implemented in our strategy with the Weighted K-means [39] that targets to balance the weights (number of points or descriptors in a region) while, at the same time, preserves the spatial proximity of centroids in a cluster.

Algorithm 4. SABES++ Region Construction

```

1: function regionConstruction( $C_c, n$ )
2:    $R \leftarrow \text{weighted-}k\text{-means}(C_c, n)$ 
3:   return  $R$ 

```

The Algorithm 4 presents the *regionConstruction* method used by SABES++ that is performed off-line as pre-processing phase. It consists of running the *weighted - k - means* method (line 2) that computes the centroid clusters or regions (R). This strategy maintains spatial proximity of the centroids in each QP. The *sendTo* and *forward* methods of SABES++ are the same used by SABES, detailed in the previous section.

5.4.1. Spatial-Aware Bucket Equal Split with replication

The strategy Spatial-Aware Bucket Equal Split with Replication (SABES+r) employs data replication as an approach to improve performance of the search. Here, data buckets may be stored in multiple QPs to reduce the number of processes consulted to answer a query. Thus, during the data distribution phase, buckets are carefully replicated into the r closest regions. The reasoning with this strategy is that buckets in near regions tend to store other buckets close in the space, which are likely to be used together to answer a query. The regions are represented by centroids, such as in SABES, and we keep for each region an ordered list of closest regions that is accessed by ‘getNNRegions’ to retrieve the closest regions.

Algorithm 5. SABES+r SendTo and Forward methods

```

1: function sendTo( $y$ )
2:    $k' \leftarrow q_c(y)$ 
3:    $\text{target} \leftarrow \text{getRegion}(R, k')$ 
4:    $\text{target} \leftarrow \text{target} \cup \text{getNNRegions}(\text{target}, R, r)$ 
5:   return  $\text{target}$ 
6: function forward( $x, w$ )
7:    $\text{nnc} \leftarrow \text{kNN}(C_c, x, w)$ 
8:   for  $i \in 1 \dots w$  do
9:      $\text{reg} \leftarrow \text{getRegion}(R, \text{nnc}[i])$ 
10:     $\text{reg} \leftarrow \text{reg} \cup \text{getNNRegions}(\text{reg}, R, r)$ 
11:    if  $\text{target} \cap \text{reg} = \emptyset$  then
12:       $\text{target} \leftarrow \text{target} \cup \text{getOne}(\text{reg})$ 
13:   return  $\text{target}$ 

```

The *sendTo* and *forward* operations are presented in Algorithm 5. The *sendTo* sends each indexed vector to r closest regions. The *forward* used in the search computes the w nearest centroids and retrieves regions in which buckets are replicated to verify if one of the regions in which the bucket is stored is already in target (line 11). If it is not, it gets one new region and adds it to target (line 12).

5.5. Data partition strategies complexity

This section summarizes the complexity of the data partitions strategies. Given an input descriptor (database vector or query vector), the complexity of *sendTo* and *forward* are computed. The complexity of the *sendTo* operation is the same for all strategies, and it is dominated by the cost of quantizing the descriptor ($q_c(y)$). After the quantization is computed the nearest coarse centroids are known and the rest of the decision regarding the QP to send the descriptor is constant in all cases. Please notice that even DES quantizes the descriptor before sending it to the QP to reduce the network bandwidth.

The DES strategy broadcasts the query to all QPs during the *forward* step, thus the destination computation is $O(1)$ and it has to send $|QPs|$ messages. Please notice that the $|QPs|$ has a correspondence with the number of computer nodes used. In our experimental evaluation, a single QP process is instantiated per node as QP is a multi-threaded

process that can efficiently use multiple CPU cores. Thus, $|QPs|$ in our case is the number of nodes that running QPs, but this correspondence may change depending on the process placement used.

Further, BES, SABES, SABES+r, and SABES++ must find the w nearest coarse centroids to the query in the *forward* in order to send messages to QPs storing them. For finding w nearest coarse centroids set, k-NN is used in the coarse centroids set, which is $O(|C_c| * D + |C_c| * \log w)$ and the rest of the function consists in mapping the coarse centroids to the QP id. This is performed in $O(1)$ for each centroid, since a mapping is available from an off-line computation in all cases, resulting in $O(w)$ for the w nearest centroids retrieved. Thus, the *forward* complexity will be $O(|C_c| * d + |C_c| * \log w + O(w))$. Also, $\min(w, |QPs|)$ message will be sent to QPs.

6. Experimental evaluation

This section evaluates the proposed strategies and optimizations for ANN execution on distribute memory machines. We used, in our experiments, a system with 160 nodes connected through a FDR Infiniband switch. Each node is equipped with 2 Intel Haswell E5-2695 v3 CPUs, 128 GB of RAM, and executes an Ubuntu server 20.04 OS. We used C++14, Intel MPI 3.1, and OpenMP 4.0 in the implementation. The IVFADC was configured with 4096 coarse centroids, $m=8$ (number of quantization subspaces), 256 centroids per sub-dimension. The dataset used has up to 16 billion 128-dimensional SIFT descriptors and 10 K queries, and was created in our previous work [18] using images collected from the web. The quality of the search results are evaluated using recall@R, the percentage or fraction of results returned ranked in the first R positions [9].

Our distribute memory scalability evaluation used a **weak scaling** setting, where the dataset indexed and searched increases proportionally to the number of computing nodes. This is more appropriate than strong scaling in our use case application, which is expected to deal with very large and growing datasets. The baseline distributed system configuration uses 1 Coordinator and 4 Query Processors with 500 million SIFT descriptors, whereas 16 billion SIFT descriptors were used with 32 Coordinators and 128 Query Processors. Each Coordinator and Query Processor instance runs exclusively in a computing node. All experiments were repeated 3 times and we observed small runtime changes with standard deviation smaller than 1.6%. Results are presented as the average of those 3 runs and a confidence interval with a confidence level of 95% was calculated. This section focus on the evaluation of distributed memory execution of IVFADC with the different data partition strategies. However, we have updated the comparison of IVFADC to state-of-the-art approaches in our target machine as is presented in [Appendix](#).

6.1. Effect of data partition strategies to performance

This section evaluates different deployments of the IVFADC for distributed memory machines. These deployments are implemented on top of the framework presented in Section 4 and differ by the data partition strategy used, as described in Section 5. The evaluation carried out here uses 40 computing nodes: 8 Coordinators and 32 Query Processors, in order to focus on the impact of the Data Partition strategies to performance. An evaluation with a larger number of nodes is presented and discussed in Section 6.2. Further, it was employed a dataset with 4 billion SIFT descriptors and 10 K queries, while IVFADC was configured with $C_c = 4096$ (coarse centroids) and $w=16$ (number of lists searched), leading to a recall@R100 of 63.7%.

The experimental results are shown in Fig. 6 for all strategies. First, it is clear that the DES strategy widely used in the application domain is the least efficient. The BES, SABES, and SABES++ have achieved speedups of, respectively, 2.3 $\times$, 3.7 $\times$, and 4.2 $\times$ as compared to DES. Also SABES+r=1 further improved SABES with gain of 1.16 $\times$ in throughput, whereas SABES+r=2 led to marginal additional improvements. In fact,

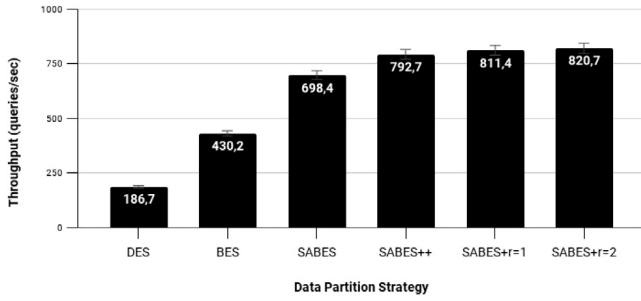


Fig. 6. Performance of IVFADC (queries/sec) as the data partition strategy used is varied in a distributed setting with 40 computing nodes.

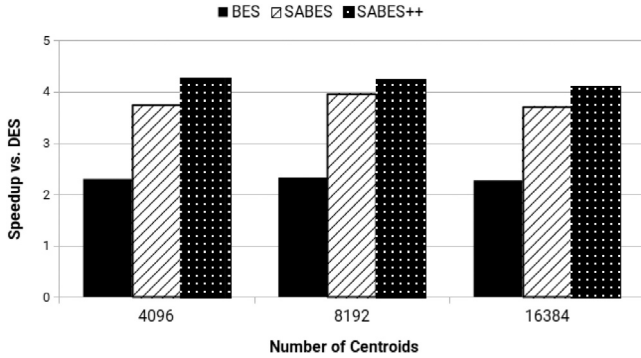


Fig. 7. Speedups attained by the different data partition strategies on top of DES as the number of centroids used is varied.

according to the 95% CI presented, SABES performance is significantly superior to BES. Also, SABES++, SABES+r=1, and SABES+r=2 attained a statistically better performance than SABES. However, there is no significant difference among SABES++, SABES+r=1, and SABES+r=2. Since the gains with replication are not significant as compared to SABES++, we believe it may not be a practical approach because we will be dealing with large datasets and replicating them 2× or 3× tends to be prohibitive in several settings.

The performance improvements achieved by BES, SABES, and SABES++ are due to a smaller number of QP processes (and nodes) that need to cooperate to answer a query as compared to DES. Because of the data partition employed, DES must forward the query to every QP that, in turn, search in their local indexes. The other strategies, on the other hand, partition the data by distributing IVFADC inverted file lists among QPs, thus only those QPs that store at least one inverted file list to be searched need to receive and process the query. Table 2 shows the number of QPs used per query (QPs/Query) for these strategies. DES uses all 32 QPs, as expected, while BES and SABES need to involve, respectively, 12.7 and 4.3 QPs on average to answer a query. This shows that the SABES strategy to distribute centroids among QPs according to their spatial locality is efficient to improve the search locality. The SABES++ does not reduce the number of QPs used as compared to SABES. Its performance improvement is a consequence of a more elaborate inverted file list distribution that reduces load imbalance among QPs. This is explained in detail in the end of this section.

The number of QPs used to answer a query impact the communication demands and associated overheads to receive and process incoming messages. Also, a smaller number QPs leads to the effect of having a larger number of descriptors into a single machine, which makes the local search more efficient. This aspect is shown in Table 2 (Total Search Time), which presents the sum of the time required by all QPs to perform the local searches. In DES, each inverted file entry is split among machines, thus it has to search in several small

Table 2

Average Number of QP processes visited and total (sum) search time of all QP processes by strategy.

Strategy	QPs per query	QP Search time (s)	Network traffic (GB)
DES	32	1344.8	15.41
BES	12.7	586.6	6.15
SABES	4.3	225.6	2.07
SABES++	4.4	230.4	2.12
SABES+r=1	3.18	189.6	1.53

Table 3

Data distribution performed by SABES and SABES++. We present the minimum and maximum number of points (descriptors) assigned to a QP and the standard deviation among all 32 QPs.

Strategy	# of QPs	Descriptors distribution		
		min	max	std
SABES	4	167078445	336612032	89578841
	32	14249858	62618917	12678393
SABES++	4	160689301	390357008	75624558
	32	21451654	46859624	4549251

sublists. Searching in a large list vs. several small ones in IVFADC is more efficient because: (i) more parallelism is available and the CPU cores are effectively used; (ii) the k-select phase that retrieves the nearest neighbors is executed a smaller number of times (one per inverted list vs. one in each QP holding a part of the inverted list). Also, the volume of information exchanged among nodes significantly reduces when fewer nodes are involved per query. In the Table 2, column GB, it is possible to observe that SABES employing 4.3 compute nodes exchanges 2.07 GB of data to process 10000 queries, while DES employing 32 nodes per query transfers 7.4× more data. These aspects improve the overall efficiency of the solution.

SABES++. Fig. 6 has shown that SABES++ is the fastest approach, achieving a speedup of 1.13× on top of SABES. Further, Table 2 shows that SABES and SABES++ visit a similar number of QPs to answer a query. The gains with SABES++ are due to its ability to balance the workload (number of descriptors) assigned to QPs, while maintaining data locality. The better data balancing of SABES++ is presented in

Table 3 that shows the maximum and minimum number of descriptors indexed by the QPs in SABES and SABES++. It can be noticed a significant reduction in data imbalance with SABES++, which reduces the overall makespan with this strategy.

We have further evaluated the relative performance of the data partition strategies for different numbers of coarse centroids (C_c) or number of IVF lists. In this case, we varied the number of coarse centroids and measured the performance of all strategies. Here, we are interested understanding if the number of coarse centroids would impact the relative performance of the data partition strategies. The results are presented in Fig. 7. As may be observed, the relative performance measured as the speedup on top of DES does not change significantly as the number of centroids used varies.

Finally, we performed an experiment with a different dataset containing 1 billion Deep Features descriptors [40] with 96 dimensions, 10k queries, and IVFADC configured with $C_c = 4096$ and $w=16$. The results are show in Fig. 8 and the recall@R100 attained is 71.9%. It is possible to observe that the behavior as partition strategy changes is similar to that seen in the SIFT dataset. However, with Deep Features dataset, the quantization process of 96 dimensional query descriptors performed on the Coordinator executes about 11% faster when compared with the same step for the 128 dimension SIFT query descriptors.

6.2. Analyzing the IVFADC scalability

This section evaluates the IVFADC performance as the number of compute nodes used is varied with different data partition strategies.

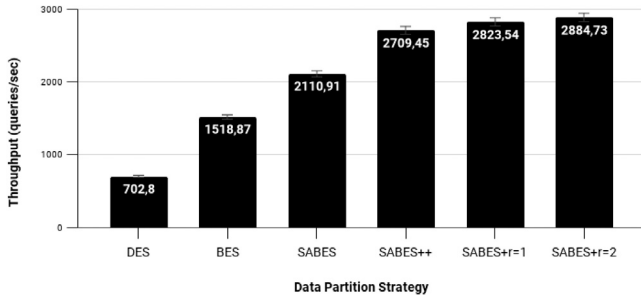


Fig. 8. Performance of IVFADC (queries/sec) as the data partition strategy used is varied in a distributed setting with 40 computing nodes with the Deep Features dataset.

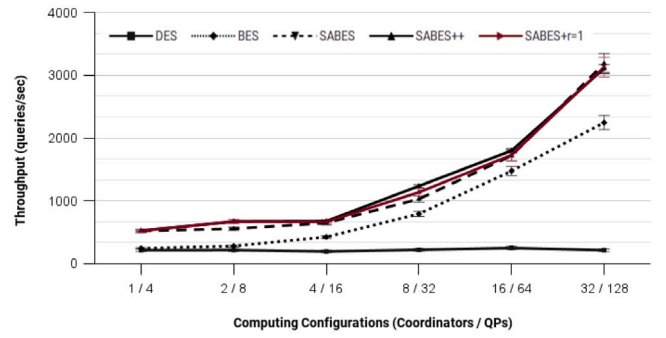
The experiment consist of a weak scaling in which dataset size and number of nodes used increase proportionally. As previously discussed, the baseline configuration uses 1 Coordinator and 4 Query Processors with 500 million SIFT descriptors, and up to 16 billion SIFT descriptors are employed here in the setting with 32 Coordinators and 128 Query Processors (160 nodes). This scalability experiment is repeated for different the number of the inverted file entries searched (w).

Fig. 9 shows the throughput (queries/sec) of different data partition strategies as the number of nodes used vary. First, it is worth noticing that DES achieved a close to linear weak scaling efficiency, whereas our BES, SABES and SABES++ reached superlinear scalability. Consequently, the differences in performance as compared to DES increases with number of nodes used.

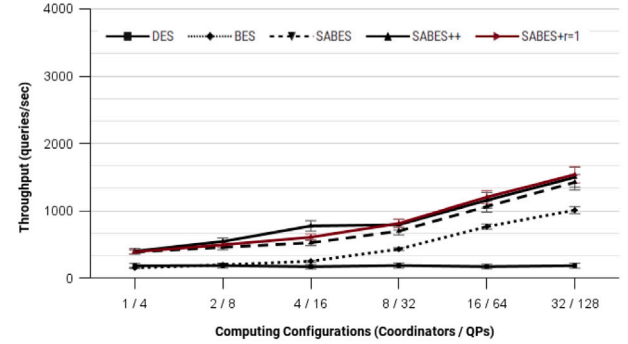
The superlinear scalability of BES, SABES and SABES++ results from them spending less time in QP local searches than DES due to aggregation of search in a few nodes. Because they distribute inverted list entries instead of partitioning every entry as performed in DES, the search for a query in BES, SABES and SABES++ involve no more than w QPs in the worst-case. This leads to a reduction in the time to process the search in QPs (presented in previous section). Further, Fig. 10 shows that the number of QPs used to answer a query grows sublinearly with the number of nodes for BES, SABES, SABES++ and SABES+r=1. For example, SABES uses 4.30 and 6.83 QPs on average to answer a query when, respectively, 40 and 160 nodes are employed. In this case, the computing resources available are 4× higher with 160, while the number of QPs involved in the search increases only 1.58×. A consequence of this aspect, a larger number of computing nodes results in a higher node local efficiency and there are more queries concurrently processed in BES, SABES and SABES++. These aspects result in superlinear scalability shown in Fig. 9. Also, the variations in w did not affected significantly the relative performance among data partition strategies, but it impacted the throughput (as expected) and the shape of the superlinear gains. Smaller w values led to stronger superlinear throughput, which was expected as the larger is w the more challenging is for the data partitioning to minimize the number of QPs used. Finally, the replication effect on the performance reduces as the number of nodes grows. It occurs because inverted file entries are spread among a larger number of QPs, making it hard for the replication to significantly reduce the number of QP visits. Consequently, for 160 nodes, for instance, there is no significant difference in performance among SABES, SABES++, and SABES+r=1 as may be seen from the 95% CI presented in Fig. 9, while those strategies still significantly better than BES.

7. Conclusions and future directions

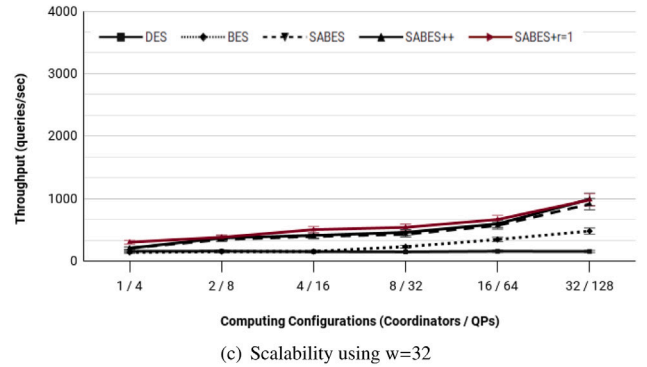
Content-Based Multimedia Retrieval (CBMR) is a core operation used in several web-based services and applications. Due to the large amount of multimedia data available, for instance, in social networks and video streaming platforms, today's CBMR applications have to



(a) Scalability using $w=8$



(b) Scalability using $w=16$



(c) Scalability using $w=32$

Fig. 9. IVFADC performance (queries/sec) in a weak scaling evaluation. The number of nodes (Coordinators/QPs) and size of dataset used increase proportionally. The baseline configuration with 5 nodes (1/4 - Coordinator/QPs) employed 500 million SIFT descriptors. The scalability evaluation is repeated for different w values.

index very large datasets while, at the same time, are submitted to a high number of simultaneous queries. For several years, CBMR systems have improved their performance with the development of efficient algorithms and indexing data structures, such as LSH [11] and Product Quantization [12], that use the computing power of a single machine. However, recent applications demands overcome the capabilities of a computing node, motivating the development CBMR indexing algorithms for distributed memory machines [9,13–18].

In this work, we have systematically investigated and evaluated distributed memory parallelization approaches available in the literature, and proposed novel and more efficient strategies. To achieve this goal, we first designed a general framework to simplify the process of deploying similarity searching algorithms on distributed memory machines. Using this framework and the IVFADC algorithm as a baseline, we have evaluated most used parallelization approach and data partition strategies (DES and BES), and have developed and deployed novel strategies (SABES and SABES++). These parallelizations of IVFADC were thoroughly evaluated at scale with up to 160 multicore computing

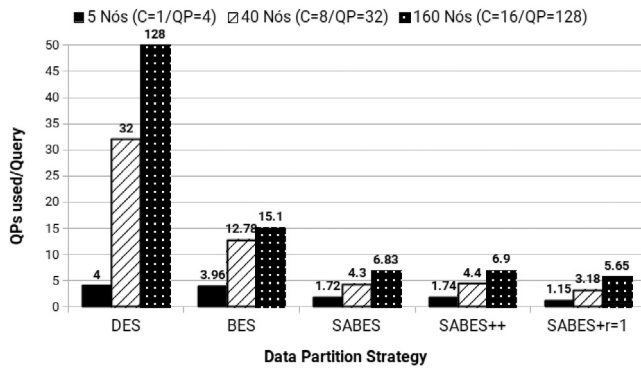


Fig. 10. Average number of QPs used to answer a query for different data partition strategies and number of computing nodes.

nodes and 16 billion SIFT descriptors. The results have demonstrated our most efficient approach (SABES++) achieved a performance improvement of 4.2 $\times$, 1.8 $\times$, respectively, on top of DES and BES. These gains are due to the ability of the proposed strategies to improve locality and, consequently, search processing efficiency on local nodes of the distributed memory machine. Further, the variation of the number of compute nodes used has shown that BES, SABES, and SABES++ have superlinear weak scaling efficiency, which in turn makes their gain over DES to increase with the number of nodes used.

We envisioned multiple directions for future work in the domain. First, we intend to integrate our distributed memory machine parallelizations with local searches (QPs) implemented on top of GPUs. We believe that the data distribution strategies proposed in this work are more appropriate to be used with that processor. Because data are located into fewer compute nodes in our approaches, the local search in each of them will be a coarse-grain task. In turn, more compute expensive node local searches should better offset GPU startup overheads and maximize the use of its parallelism. We also want to extend the SABES++ with other strategies able to consider data locality and load balancing together. Finally, we want to leverage the solutions and data partition strategies developed in this work to implement and evaluate other ANN algorithms. While the strategies have been developed into a framework to facilitate their reuse, our current work has only implemented a single ANN searching algorithm: IVFADC.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.parco.2022.102992>.

Data availability

Data will be made available on request.

Acknowledgments

This work was partially funded by INWeb, MASWeb, CAPES, Brazil, CNPq, Brazil, Finep, Brazil, PRPq/UFMG, Brazil, and Fapemig, Brazil. This work used the Extreme Science and Engineering Discovery Environment (XSEDE), which is supported by National Science Foundation (NSF), USA grant number ACI-1548562. Specifically, it used the Bridges system, which is supported by NSF, USA award number ACI-1445606, at the Pittsburgh Supercomputing Center (PSC).

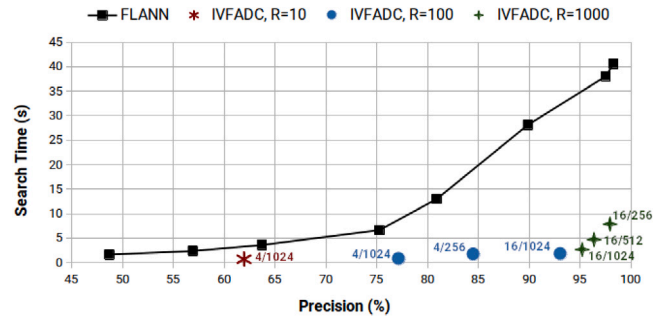


Fig. A.1. Comparison of IVFADC and FLANN for different search precisions (1-recall@1).

Appendix. IVFADC vs FLANN and exact search

This section evaluates the IVFADC performance. IVFADC was already compared to FLANN in [12], but we have chosen to update the results using our target machine and include the exhaustive k-NN search as a reference. FLANN is an interesting baseline as it implements multiple ANN strategies and chooses the best one for a given problem (dataset). The algorithm selected by FLANN in this case was the hierarchical k-means tree. The exact k-NN was implemented using the BLAS/LAPACK underneath to assert performance, and serves as a reference to evaluate exact and approximate search trade-offs. We used a dataset with 1 million SIFT descriptors and 10 K queries in this experiment. Quality was measured as 1-recall@1 or the percentage of query results in which the nearest neighbor is ranked first (also called precision). Experiments were executed sequentially using a single CPU core. FLANN automatically selects the parameters for each given target precision, while we varied w and C_c in IVFADC (shown as $w/\#$ of coarse centroids (C_c in Fig. A.1) along with R (size of returned list).

The results shown in Fig. A.1. For the same level of precision, IVFADC is significantly faster than FLANN. Also, IVFADC uses 26 MB of RAM, while FLANN has required about 600 MB of memory. The smaller memory footprint is another key attribute of IVFADC to enable indexing very large datasets. The exhaustive k-NN took 57 s to execute, which is much slower than IVFADC even when the latter is configured to attain high precision (e.g., about 95%).

References

- [1] C. Böhm, S. Berchtold, D.A. Keim, Searching in high-dimensional spaces: Index structures for improving the performance of multimedia databases, *ACM Comput. Surv.* 33 (2001) 322–373, <http://dx.doi.org/10.1145/502807.502809>.
- [2] N. Sundaram, A. Turmukhametova, N. Satish, T. Mostak, P. Indyk, S. Madden, P. Dubey, Streaming Similarity Search over One Billion Tweets Using Parallel Locality-Sensitive Hashing, *Proc. VLDB Endow.* 6 (14) (2013) 1930–1941, <http://dx.doi.org/10.14778/2556549.2556574>.
- [3] J. Wan, D. Wang, S.C.H. Hoi, P. Wu, J. Zhu, Y. Zhang, J. Li, Deep Learning for Content-Based Image Retrieval: A Comprehensive Study, in: *Proceedings of the 22Nd ACM International Conference on Multimedia, MM '14*, ACM, New York, NY, USA, 2014, pp. 157–166, <http://dx.doi.org/10.1145/2647868.2654948>.
- [4] Y. Gong, L. Wang, R. Guo, S. Lazebnik, Multi-scale Orderless Pooling of Deep Convolutional Activation Features, in: D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars (Eds.), *Computer Vision – ECCV 2014*, Springer International Publishing, Cham, 2014, pp. 392–407, http://dx.doi.org/10.1007/978-3-319-10584-0_26.
- [5] M. Douze, H. Jégou, H. Sandhawalia, L. Amsaleg, C. Schmid, Evaluation of GIST Descriptors for Web-scale Image Search, in: *Proceedings of the ACM International Conference on Image and Video Retrieval, CIVR '09*, ACM, New York, NY, USA, 2009, pp. 19:1–19:8, <http://dx.doi.org/10.1145/1646396.1646421>.
- [6] H. Jégou, F. Perronnin, M. Douze, J. Sánchez, P. Pérez, C. Schmid, Aggregating Local Image Descriptors into Compact Codes, *IEEE Trans. Pattern Anal. Mach. Intell.* 34 (9) (2012) 1704–1716, <http://dx.doi.org/10.1109/TPAMI.2011.235>.
- [7] J.H. Friedman, J.L. Bentley, R.A. Finkel, An algorithm for finding best matches in logarithmic expected time, *ACM Trans. Math. Software* 3 (3) (1977) 209–226, <http://dx.doi.org/10.1145/355744.355745>.

- [8] A. Beygelzimer, S. Kakade, J. Langford, Cover Trees for Nearest Neighbor, in: Proceedings of the 23rd International Conference on Machine Learning, ICML '06, 2006, pp. 97–104, <http://dx.doi.org/10.1145/1143844.1143857>.
- [9] M. Muja, D.G. Lowe, Scalable Nearest Neighbor Algorithms for High Dimensional Data, *IEEE Trans. Pattern Anal. Mach. Intell.* 36 (11) (2014) 2227–2240, <http://dx.doi.org/10.1109/TPAMI.2014.2321376>.
- [10] R. Weber, H.-J. Schek, S. Blott, A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces, in: Proceedings of the 24rd International Conference on Very Large Data Bases, VLDB '98, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1998, pp. 194–205, <http://dx.doi.org/10.5555/645924.671192>.
- [11] A. Gionis, P. Indyk, R. Motwani, Similarity Search in High Dimensions via Hashing, in: Proceedings of the 25th International Conference on Very Large Data Bases, VLDB '99, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1999, pp. 518–529, <http://dx.doi.org/10.5555/645925.671516>.
- [12] H. Jegou, M. Douze, C. Schmid, Product Quantization for Nearest Neighbor Search, *IEEE Trans. Pattern Anal. Mach. Intell.* 33 (1) (2010) 117–128, <http://dx.doi.org/10.1109/TPAMI.2010.57>.
- [13] A. Stupar, S. Michel, R. Schenkel, RankReduce - processing K-Nearest Neighbor queries on top of MapReduce, in: Proceedings of the 8th Workshop on Large-Scale Distributed Systems for Information Retrieval (LSDS-IR'10), 2010, pp. 1–6, URL <http://ceur-ws.org/Vol-630/lssir2.pdf>.
- [14] B. Bahmani, A. Goel, R. Shinde, Efficient Distributed Locality Sensitive Hashing, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, CIKM, 2012, pp. 2174–2178, <http://dx.doi.org/10.1145/2396761.2398596>.
- [15] D. Moise, D. Shestakov, G. Gudmundsson, L. Amsaleg, Indexing and Searching 100M Images with Map-reduce, in: Proceedings of the 3rd ACM Conference on International Conference on Multimedia Retrieval, ICMR '13, 2013, pp. 17–24, <http://dx.doi.org/10.1145/2461466.2461470>.
- [16] M. Kruliš, T. Skopal, J. Lokoč, C. Beecks, Combining CPU and GPU architectures for fast similarity search, *Distribut. Parallel Databases* 30 (3) (2012) 179–207, <http://dx.doi.org/10.1007/s10619-012-7092-4>.
- [17] G. Teodoro, E. Valle, N. Mariano, R. Torres, W. Meira, J.H. Saltz, Approximate similarity search for online multimedia services on distributed CPU-GPU platforms, *VLDB J.* 23 (3) (2014) 427–448, <http://dx.doi.org/10.1007/s00778-013-0329-7>.
- [18] G. Andrade, A. Fernandes, J.M. Gomes, R. Ferreira, G. Teodoro, Large-scale parallel similarity search with Product Quantization for online multimedia services, *J. Parallel Distrib. Comput.* 125 (2019) 81–92, <http://dx.doi.org/10.1016/j.jpdc.2018.11.009>.
- [19] G. Andrade, G. Teodoro, R. Ferreira, Scalable and Efficient Spatial-Aware Parallelization Strategies for Multimedia Retrieval, in: 32nd IEEE International Symposium on Computer Architecture and High Performance Computing, SBAC-PAD 2020, Porto, Portugal, September 9–11, 2020, IEEE, 2020, pp. 124–131, <http://dx.doi.org/10.1109/SBAC-PAD49847.2020.00027>.
- [20] J. Dean, S. Ghemawat, MapReduce: simplified data processing on large clusters, *Commun. ACM* 51 (1) (2008) 107–113, <http://dx.doi.org/10.1145/1327452.1327492>.
- [21] A. Toshniwal, S. Taneja, A. Shukla, K. Ramasamy, J.M. Patel, S. Kulkarni, J. Jackson, K. Gade, M. Fu, J. Donham, N. Bhagat, S. Mittal, D. Ryaboy, Storm@twitter, in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, SIGMOD '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 147–156, <http://dx.doi.org/10.1145/2588555.2595641>.
- [22] G.T. Gudmundsson, B.T. Jónsson, L. Amsaleg, M.J. Franklin, Prototyping a Web-Scale Multimedia Retrieval Service Using Spark, *ACM Trans. Multimedia Comput. Commun. Appl.* 14 (3s) (2018) <http://dx.doi.org/10.1145/3209662>.
- [23] M. Zaharia, M. Chowdhury, M.J. Franklin, S. Shenker, I. Stoica, Spark: Cluster Computing with Working Sets, in: Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing, HotCloud '10, 2010, p. 10.
- [24] M.P. Forum, MPI: A Message-Passing Interface Standard, Tech. rep., University of Tennessee, USA, 1994.
- [25] R. Ferreira, W.M. Jr., D.O.G. Neto, L.M. de A. Drummond, B. Coutinho, G. Teodoro, T. Tavares, R.B. Araújo, G.T. Ferreira, Anthill: A Scalable Run-Time Environment for Data Mining Applications, in: 17th Symposium on Computer Architecture and High Performance Computing (SBAC-PAD 2005), 24–27 October 2005, Rio de Janeiro, Brazil, IEEE Computer Society, 2005, pp. 159–167, <http://dx.doi.org/10.1109/CAHPC.2005.12>.
- [26] R. Souza, A. Fernandes, T.S.F.X. Teixeira, G. Teodoro, R. Ferreira, Online multimedia retrieval on CPU-GPU platforms with adaptive work partition, *J. Parallel Distributed Comput.* 148 (2021) 31–45, <http://dx.doi.org/10.1016/j.jpdc.2020.10.001>.
- [27] N. Meisburger, A. Shrivastava, Distributed tera-scale similarity search with MPI: provably efficient similarity search over billions without a single distance computation, 2020, CoRR abs/2008.03260 [arXiv:2008.03260](https://arxiv.org/abs/2008.03260).
- [28] V. Gil-Costa, M. Marin, Load Balancing Query Processing in Metric-Space Similarity Search, in: 2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (Ccgird 2012), 2012, pp. 368–375, <http://dx.doi.org/10.1109/CCGrid.2012.30>.
- [29] P. Ram, D. Lee, H. Ouyang, A.G. Gray, Rank-Approximate Nearest Neighbor Search: Retaining Meaning and Speed in High Dimensions, in: Y. Bengio, D. Schuurmans, J.D. Lafferty, C.K.I. Williams, A. Culotta (Eds.), *Advances in Neural Information Processing Systems 22*, Curran Associates, Inc., 2009, pp. 1536–1544.
- [30] M. Muja, D. Lowe, Fast approximate nearest neighbors with automatic algorithm configuration., in: VISAPP 2009 - Proceedings of the 4th International Conference on Computer Vision Theory and Applications, vol. 1, 2009, pp. 331–340.
- [31] E. Valle, M. Cord, S. Philipp-Foliguet, High-Dimensional Descriptor Indexing for Large Multimedia Databases, in: Proceedings of the 17th ACM Conference on Information and Knowledge Management, CIKM '08, Association for Computing Machinery, New York, NY, USA, 2008, pp. 739–748, <http://dx.doi.org/10.1145/1458082.1458181>.
- [32] C. Silpa-Anan, R. Hartley, Optimised KD-trees for fast image descriptor matching, in: 2008 IEEE Conference on Computer Vision and Pattern Recognition, 2008, <http://dx.doi.org/10.1109/CVPR.2008.4587638>.
- [33] D. Nister, H. Stewenius, Scalable recognition with a vocabulary tree, in: 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), 2, 2006, pp. 2161–2168, <http://dx.doi.org/10.1109/CVPR.2006.264>.
- [34] J. Johnson, M. Douze, H. Jegou, Billion-scale similarity search with GPUs, 2017, arXiv preprint [arXiv:1702.08734](https://arxiv.org/abs/1702.08734).
- [35] X. Tang, M. Alabduljalil, X. Jin, T. Yang, Load Balancing for Partition-Based Similarity Search, in: Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 193–202, <http://dx.doi.org/10.1145/2600428.2609624>.
- [36] M.A. Alabduljalil, X. Tang, T. Yang, Optimizing Parallel Algorithms for All Pairs Similarity Search, in: Proceedings of the Sixth ACM International Conference on Web Search and Data Mining, WSDM '13, Association for Computing Machinery, New York, NY, USA, 2013, pp. 203–212, <http://dx.doi.org/10.1145/2433396.2433422>.
- [37] E. Chávez, G. Navarro, A Compact Space Decomposition for Effective Metric Indexing, *Pattern Recognit. Lett.* 26 (9) (2005) 1363–1376, <http://dx.doi.org/10.1016/j.patrec.2004.11.014>.
- [38] X. Zhao, J. Zhang, X. Qin, k NN-DP: Handling data skewness in kNN joins using MapReduce, *IEEE Trans. Parallel Distrib. Syst.* 29 (3) (2018) 600–613, <http://dx.doi.org/10.1109/TPDS.2017.2767596>.
- [39] K. Kerdprasop, N. Kerdprasop, P. Sattayatham, Weighted K-Means for Density-Biased Clustering, in: A.M. Tjoa, J. Trujillo (Eds.), *Data Warehousing and Knowledge Discovery*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 488–497, http://dx.doi.org/10.1007/11546849_48.
- [40] A.B. Yandex, V. Lempitsky, Efficient indexing of billion-scale datasets of deep descriptors, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2016, pp. 2055–2063, <http://dx.doi.org/10.1109/CVPR.2016.226>.